

Model registry

# Remeber

- The four core components of MLflow are **MLflow Tracking**, **MLflow Projects**, **MLflow Models**, and the **MLflow Model Registry**.
- These tools help manage the entire machine learning life cycle from code creation to production testing

# Why Model registry

- machine learning, **managing models effectively** is crucial for production success
- As teams collaborate on ML projects, tracking different model versions, understanding their performance, and deploying the right models becomes **increasingly complex**
- This is where **MLflow Model Registry** comes in — a **centralized system for managing** the complete **lifecycle** of ML models.

# Topics to be discuss

- What is Model Registry?
- Setting Up the Environment
- Registering a Model
- Registering Using SDK
- Creating Model Versions
- Updating Model Metadata
- Getting Model Versions
- Deleting Information or Metadata

# What is Model Registry?

- **MLflow Model Registry** is a centralized model store with **APIs and UI** support to collaboratively manage the full lifecycle of ML models from development to production.
- We can think of it as a version control system specifically designed for machine learning models.
- **Key Benefits**
  - Centralized Management
  - Version Control
  - Lifecycle Management
  - Deployment Readiness
  - Collaboration

# Core Concepts

- **Registered Models:** Named entities that represent a collection of model versions. They act as containers for versions of the same model over time.
- **Model Versions:** Each time we register a model, MLflow creates a new version (v1, v2, v3, ...). Versions maintain:
  - - Link to the source MLflow run
  - - Artifact location
  - - Version-specific metadata and tags
  - - Status (None, Archived, Production)

- **Tags:** Key-value pairs for metadata organization:
- **Registered Model Tags:** Apply to all versions (e.g., `framework`, `task`, `project\_name`)
- - **Version Tags:** Specific to individual versions (e.g., `validation\_status`, `deployment\_date`)

# Setting Up the Environment

- **Prerequisites**

- - Python 3.9 or higher
- - Linux/macOS/WSL or Windows PowerShell
- - Basic understanding of machine learning and Python



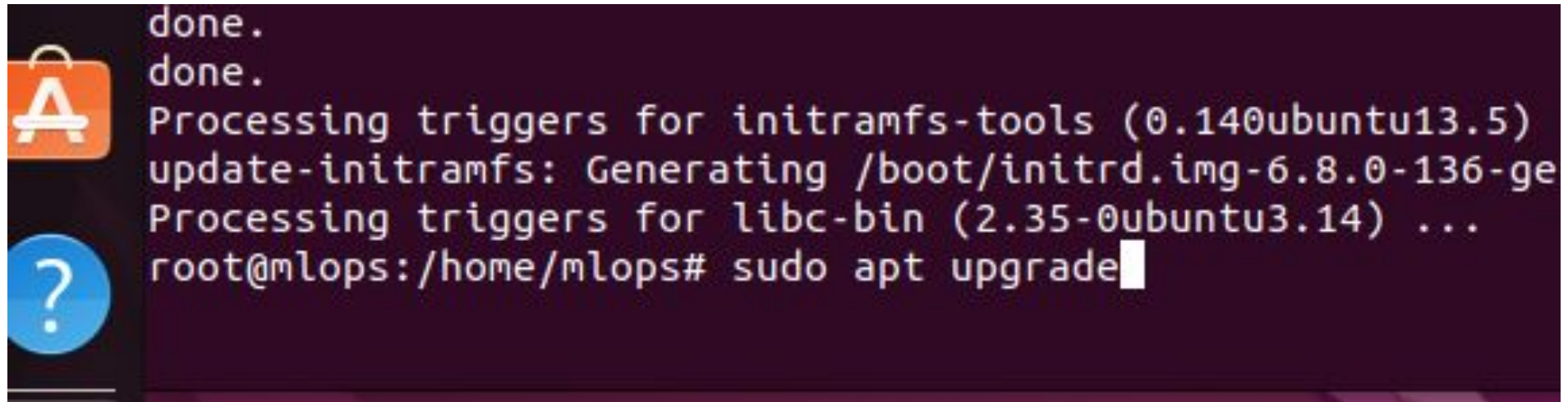
# Installation

- First, update our system and install dependencies:
- # For Ubuntu/Debian  
sudo apt update  
sudo apt upgrade  
sudo apt install -y python3 python3-pip python3-venv python3-full

# update

```
tadata [127 kB]  
Fetched 30.1 MB in 11s (2,763 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
211 packages can be upgraded. Run 'apt list --upgr  
root@mlops:/home/mlops# sudo apt update
```

# Upgrade



A terminal window with a dark purple background. On the left side, there is a vertical dock with two icons: an orange square with a white 'A' (Ubuntu Software Center) and a blue circle with a white question mark. The terminal text is as follows:

```
done.  
done.  
Processing triggers for initramfs-tools (0.140ubuntu13.5)  
update-initramfs: Generating /boot/initrd.img-6.8.0-136-ge  
Processing triggers for libc-bin (2.35-0ubuntu3.14) ...  
root@mlops:/home/mlops# sudo apt upgrade
```

# Create a virtual environment and install required packages:

- `python3 -m venv .venv`  
`source .venv/bin/activate` # On Windows: `.\.venv\Scripts\Activate.ps1`

# Install dependencies

`pip install --upgrade pip`

`pip install mlflow scikit-learn pandas matplotlib seaborn`

# Or install from requirements.txt:

- `pip install -r requirements.txt`

# Starting MLflow Server

- To access the MLflow UI, you need to start the MLflow tracking server. Open a terminal and run:
- `mlflow server --host 0.0.0.0 --port 5000 --allowed-hosts '*' --cors-allowed-origins '*'`

```
root@mlops:/home/mlops# mlflow ui
Backend store URI not provided. Using sqlite:///mlflow.db
Registry store URI not provided. Using backend store URI.
2026/08/02 13:28:29 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/08/02 13:28:29 INFO mlflow.store.db.utils: Updating database tables
[MLflow] Security middleware enabled with default settings (localhost-only). To
allow connections from other hosts, use --host 0.0.0.0 and configure --allowed-origins and --cors-allowed-origins.
2026/08/02 13:28:33 INFO:      Unicorn running on http://127.0.0.1:5000 (Press
RL+C to quit)
2026/08/02 13:28:33 INFO:      Started parent process [34961]
```

# MLFLOW UI

Firefox Web Browser

Experiments - MLflow

localhost:5000/#/experiments

mlflow 3.15.0

GenAI Model training

Home

Experiments

Prompts

MCP registry Beta

AI Gateway New

## Experiments

Create Compare Delete

Search experi... Q Tag v

| <input type="checkbox"/> | Name    | Time created       | Last modifi... ⌵   | Description | Tags |
|--------------------------|---------|--------------------|--------------------|-------------|------|
| <input type="checkbox"/> | Default | 08/02/2026, 01:... | 08/02/2026, 01:... | -           |      |

< Previous Next >

25 / page v



# Registering a Model

- Registering with UI
- we will create a run of a Random Forest Classifier model and will register it on Mlflow UI.

Models - MLflow

localhost:5000/#/models

mlflow

3.15.0

GenAI

Model training

Home

Experiments

Model registry

Registered Models

Search registered models

Name

Latest version

Aliased versions

Created by

Share and manage machine learning models

+ Create model

## Create Model



Model name

After creation, you can register logged models as new versions. [Learn more.](#)

Cancel

Create

+ Create model



Model: Reg\_tesr\_model - ×



localhost:5000/#/models/Reg\_tesr\_model

mlflow

3.15.0



GenAI

Model training



Home



Experiments



Model registry

[Registered Models](#) >

**Reg\_tesr\_model**

Created Time: 08/02/2026, 01:36:01 PM

Last Modified: 08/02/2026, 01:36:01 PM

> Description [Edit](#)

▼ Tags

Name

Value

Actions

No tags found.

Name

Value

Add

▼ Versions

Compare

Version

Registered at ↕

Created by

Tags

# Registering with Code

- Let's start with the simplest approach — registering a model during a training run. We'll use a **Random Forest Classifier on the Iris dataset**.
- **Basic Registration**
- Here's a complete example from our **rf\_single\_file\_mlflow.py**:

- ```
import mlflow
from mlflow.tracking import MlflowClient
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

TRACKING_URI = "http://localhost:5000"
REGISTERED_MODEL_NAME = "random-forest-classifier"

mlflow.set_tracking_uri(TRACKING_URI)
client = MlflowClient()

# Load and split data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- # Train model  
clf = RandomForestClassifier(n\_estimators=120, random\_state=42)  
clf.fit(X\_train, y\_train)  
  
# Log model and auto-register  
with mlflow.start\_run(run\_name="rf-iris-single-file") as run:  
mlflow.log\_param("n\_estimators", 120)  
mlflow.log\_param("random\_state", 42)  
  
y\_pred = clf.predict(X\_test)  
acc = accuracy\_score(y\_test, y\_pred)  
mlflow.log\_metric("accuracy", acc)  
  
# This automatically registers the model!  
mlflow.sklearn.log\_model(  
sk\_model=clf,  
artifact\_path="RandomForestClassifier",  
registered\_model\_name=REGISTERED\_MODEL\_NAME  
)

- **What happened?**

- - MLflow logged the model artifact to the run
- - Created a registered model named `random-forest-classifier` (if it didn't exist)
- - Created version 1 under this registered model
- - All in one step!
- We can verify this in the MLflow UI by navigating to **Models** and seeing our registered model.



- **SDK Auto-Registration**

- In our `sdk_registry.py` file, we demonstrate this:

- ```
import mlflow
from mlflow.tracking import MlflowClient
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

TRACKING_URI = "http://localhost:5000"
MODEL_SDK_NAME = "registered-model-sdk"

mlflow.set_tracking_uri(TRACKING_URI)
client = MlflowClient()

def train_fit_model(n_estimators=50, random_state=42):
    """Train a RandomForestClassifier model."""
    X, y = load_iris(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=random_state
    )
```

- ```
model = RandomForestClassifier(n_estimators=n_estimators,  
random_state=random_state)  
model.fit(X_train, y_train)  
return model
```

```
def sdk_auto_register():  
    """Register a model using SDK auto-register via log_model."""  
    model = train_fit_model(80)  
    with mlflow.start_run(run_name="sdk-auto-register") as run:  
        mlflow.sklearn.log_model(  
            sk_model=model,  
            artifact_path="RandomForestClassifier",  
            registered_model_name=MODEL_SDK_NAME # This parameter does the magic!  
        )  
        print(f"[SDK] Auto-registered under '{MODEL_SDK_NAME}' with  
run_id={run.info.run_id}")  
    return run.info.artifact_uri, run.info.run_id
```

- **Key Point:** By adding  
`registered\_model\_name="registered-model-sdk"` to `log\_model()`,  
MLflow automatically:
  - 1. Creates the registered model if it doesn't exist
  - 2. Creates version 1 (or next version if the model exists)
  - 3. Links the version to the current run

- This is the **recommended approach** for most use cases because it's simple and requires minimal code.

- **Creating Registered Models via Client**
- For more control, We can explicitly create registered models using ``MlflowClient``:

- `from mlflow.tracking import MlflowClient`

```
client = MlflowClient()  
model_name = "registered-model-via-client"
```

```
try:  
    result = client.create_registered_model(name=model_name)  
    print(f"Model {result.name} created")  
    print(f"Model description: {result.description}")  
    print(f"Model creation timestamp: {result.creation_timestamp}")  
    print(f"Model tags: {result.tags}")  
except Exception as e:  
    print(f"Error: {e}")
```

- This is useful when we want to:
  - - Create a registered model before training
  - - Set up the model structure before adding versions
  - - Have more explicit control over the registration process



- **Creating Model Versions**

- Once we have a registered model, we can create multiple versions. Each version represents a different iteration of our model.

- **Creating Versions from Existing Runs**

- In our `client_version_tag_registry.py`, we show how to create versions from existing runs:

- ```
def create_model_version_from_client(name: str, source: str, run_id: str):  
    """Create a model version using MlflowClient."""  
    # Ensure the registered model exists  
    try:  
        client.get_registered_model(name)  
    except Exception:  
        client.create_registered_model(name)  
  
    # Create a version from an existing run  
    mv = client.create_model_version(  
        name=name,  
        source=source, # Artifact URI from the run  
        run_id=run_id,  
        description="Model version created using MlflowClient"  
    )  
    print(f"[Client] Created model version v{mv.version} for '{name}'")  
    return mv
```

- # After running a training run  
with `mlflow.start_run(run_name="my-training-run")` as run:  
`mlflow.sklearn.log_model(sk_model=model, artifact_path="model")`  
`artifact_uri = run.info.artifact_uri`  
`run_id = run.info.run_id`  
  
# Create a version from that run  
`mv = create_model_version_from_client(  
name="my-model",  
source=artifact_uri,  
run_id=run_id  
)`  
  
`print(f"Version: {mv.version}")  
print(f"Status: {mv.status}")  
print(f"Source: {mv.source}")  
print(f"Run ID: {mv.run_id}")`

- ```

def canonical_flow_with_versions_tags_aliases():
    """Complete workflow with versions, tags, and aliases."""
    registered_model_name = "random-forest-classifier-lab"

    # Ensure registered model exists
    try:
        client.get_registered_model(registered_model_name)
    except Exception:
        client.create_registered_model(registered_model_name)

    # Version A
    modelA = train_fit_model(60)
    with mlflow.start_run(run_name="canonical-A") as runA:
        mlflow.sklearn.log_model(
            sk_model=modelA,
            artifact_path="RandomForestClassifier",
            registered_model_name=registered_model_name
        )
    runA_id = runA.info.run_id

    # Update registered model description
    client.update_registered_model(
        name=registered_model_name,
        description="Canonical RF classifier lab model"
    )

    # Find version A and update it
    versions_A = client.search_model_versions(
        f"name = '{registered_model_name}' and run_id = '{runA_id}'"
    )
    vA = versions_A[0].version

    client.update_model_version(
        name=registered_model_name,
        version=vA,
        description="Version A: n_estimators=60"
    )

```

- ```

# Set registered model tags
for k, v in {
  "project_name": "UNDEFINED",
  "task": "Classification",
  "framework": "sklearn"
}.items():
  client.set_registered_model_tag(
    name=registered_model_name,
    key=k,
    value=v
  )

# Set aliases for version A
for alias in ["development", "candidate", "Champion"]:
  client.set_registered_model_alias(
    name=registered_model_name,
    alias=alias,
    version=vA
  )

# Version B
modelB = train_fit_model(120)
with mlflow.start_run(run_name="canonical-B") as runB:
  mlflow.sklearn.log_model(
    sk_model=modelB,
    artifact_path="RandomForestClassifier"
  )
  mvB = client.create_model_version(
    name=registered_model_name,
    source=runB.info.artifact_uri,
    run_id=runB.info.run_id,
    description="Version B: n_estimators=120"
  )

# Move Champion alias to version B
client.set_registered_model_alias(
  name=registered_model_name,
  alias="Champion",
  version=mvB.version
)

```

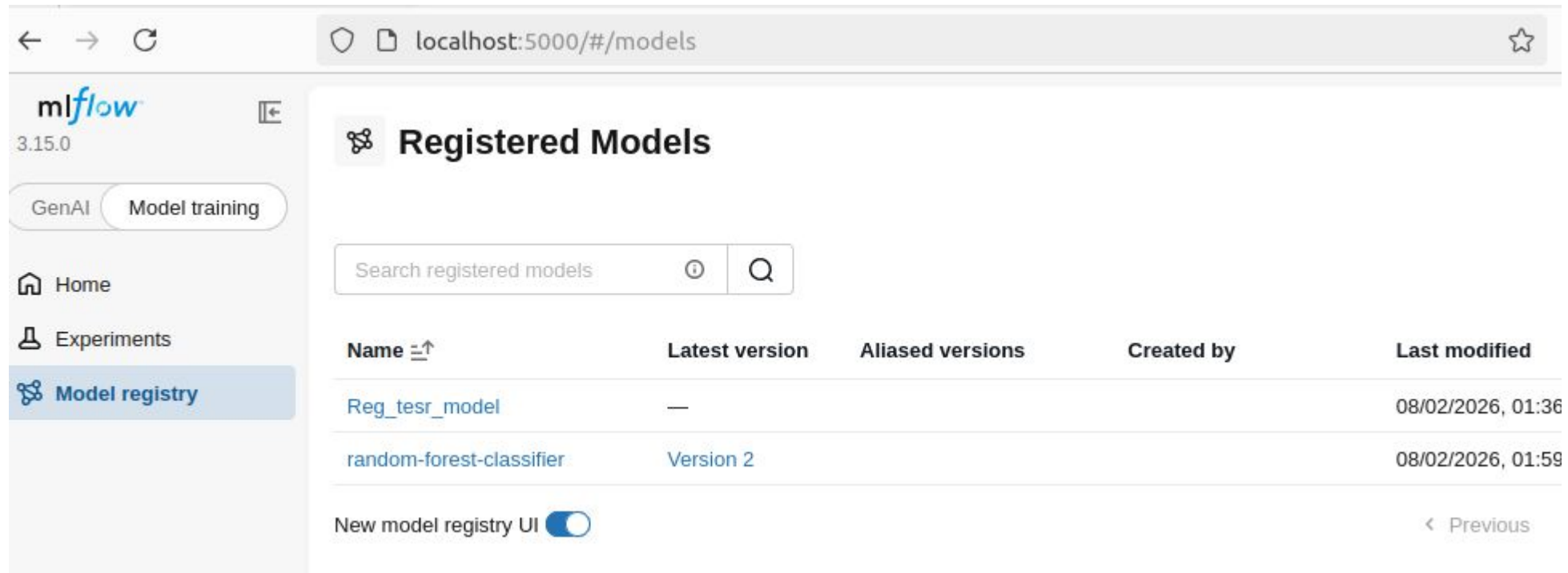
```
view run file at: http://localhost:5000/#/experiments/0/runs/23c13dd5bf0a41c4a7d52e491388bd4f
View experiment at: http://localhost:5000/#/experiments/0
mlops@mlops:~$
mlops@mlops:~$ ls
client_version_tag_registry.py  mlflow.db  sdk_registry.py
Desktop                        Music      snap
Documents                     Pictures   Templates
Downloads                     Public     Videos
mlartifacts                   rf_single_file_mlflow.py
mlops@mlops:~$ python sdk_registry.py
mlops@mlops:~$
```

```
mlops@mlops:~$ python rf_single_file_mlflow.py
2026/08/02 13:56:18 WARNING mlflow.models.model: `artifact_path` is deprecated.
Please use `name` instead.

Successfully registered model 'random-forest-classifier'.
2026/08/02 13:57:02 INFO mlflow.store.model_registry.abstract_store: Waiting up
to 300 seconds for model version to finish creation. Model name: random-forest-c
lassifier, version 1
Created version '1' of model 'random-forest-classifier'.
🔗 View run rf-iris-single-file at: http://localhost:5000/#/experiments/0/runs/2
3c13dd5bf0a41c4a7d52e491388bd4f
🔗 View experiment at: http://localhost:5000/#/experiments/0
mlops@mlops:~$
mlops@mlops:~$ ls
client_version_tag_registry.py  mlflow.db  sdk_reristry.py
Desktop                        Music      snap
Documents                     Pictures  Templates
Downloads                     Public    Videos
mlartifacts                   rf_single_file_mlflow.py
mlops@mlops:~$ python sdk_reristry.py
mlops@mlops:~$ python client_version_tag_registry.py
mlops@mlops:~$ mlflow ui
```



# Registered Model



The screenshot displays the mlflow Model Registry interface in a web browser. The browser's address bar shows the URL `localhost:5000/#/models`. The mlflow logo and version `3.15.0` are in the top left. A sidebar on the left contains navigation links: `Home`, `Experiments`, and `Model registry` (which is highlighted). Above the sidebar, there are tabs for `GenAI` and `Model training`. The main content area is titled `Registered Models` and includes a search bar labeled `Search registered models`. Below the search bar is a table with the following columns: `Name`, `Latest version`, `Aliased versions`, `Created by`, and `Last modified`. The table lists two models: `Reg_tesr_model` and `random-forest-classifier`. At the bottom of the interface, there is a toggle switch for `New model registry UI` and a `< Previous` link.

| Name                                     | Latest version | Aliased versions | Created by | Last modified     |
|------------------------------------------|----------------|------------------|------------|-------------------|
| <a href="#">Reg_tesr_model</a>           | —              |                  |            | 08/02/2026, 01:36 |
| <a href="#">random-forest-classifier</a> | Version 2      |                  |            | 08/02/2026, 01:59 |

# Version change

[Registered Models](#) >

## random-forest-classifier



Created Time: 08/02/2026, 01:57:01 PM

Last Modified: 08/02/2026, 01:59:59 PM

> [Description](#) [Edit](#)

> [Tags](#)

▼ [Versions](#) [Compare](#)

New model registry UI ☒

| Version                   | Registered at           | Created by | Tags                | Aliases             | Description |
|---------------------------|-------------------------|------------|---------------------|---------------------|-------------|
| <a href="#">Version 2</a> | 08/02/2026, 01:59:59 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |             |
| <a href="#">Version 1</a> | 08/02/2026, 01:57:01 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |             |

< [Previous](#) [Next](#) >

25 / page ▼

Registered Models >

# random-forest-classifier

Created Time: 08/02/2026, 01:57:01 PM

Last Modified: 08/02/2026, 02:04:13 PM

> Description [Edit](#)

> Tags

▼ Versions 

Compare

|   | Version                   | Registered at ⌵         | Created by | Tags                | Aliases             |
|---|---------------------------|-------------------------|------------|---------------------|---------------------|
| ✓ | <a href="#">Version 3</a> | 08/02/2026, 02:04:13 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |
| ✓ | <a href="#">Version 2</a> | 08/02/2026, 01:59:59 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |
| ✓ | <a href="#">Version 1</a> | 08/02/2026, 01:57:01 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |

 3.15.0

GenAI

Model training

←

Default

☰

Runs

🔗

Models

🔗

Traces

🔗

Model registry

## Registered Models

Search registered models

ⓘ

🔍

| Name  | Latest version | Aliased versions | Created by | Last m  |
|----------------------------------------------------------------------------------------|----------------|------------------|------------|---------|
| <a href="#">Reg_test_model</a>                                                         | Version 1      |                  |            | 08/02/2 |
| <a href="#">random-forest-classifier</a>                                               | Version 3      |                  |            | 08/02/2 |

New model registry UI 

← P



GenAI

Model training

Default

**Runs**

Models

Traces

Model registry

Experiments &gt; Default &gt;

**Runs** ⓘ

Views ▾



⌚ All time ▾



+ New run

State: Active ▾

Datasets ▾

⇅ Sort: Created ▾

Columns ▾

Group by ▾

| <input type="checkbox"/> | Run Name              | Created ⇅        | Dataset | Duration | Source      | Models |
|--------------------------|-----------------------|------------------|---------|----------|-------------|--------|
| <input type="checkbox"/> | ● rf-iris-single-file | ✓ 11 minutes ago | -       | 46.3s    | rf_singl... | Reg_   |



## rf-iris-single-file

Overview

**Model metrics**

System metrics

Traces

Artifacts



Search metric charts



**Model metrics (1)**



**accuracy**





# RandomForestClassifier

Overview Traces Artifacts

About this logged model

|               |                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Created at    | 13 minutes ago                                                                                                                      |
| Created by    | mlops                                                                                                                               |
| Status        |  Ready                                             |
| Model ID      | m-fdaf5024330446c88868f155383f5747               |
| Source run    | <a href="#">rf-iris-single-file</a>                                                                                                 |
| Source run ID | <a href="#">23c13dd5bf0a41c4a7d52e491388bd4f</a>  |
| Logged from   |  rf_single_file_mlflow.py                          |

Datasets used

None

Model versions

 random-forest-classifier v1

3.15.0

GenAI

Model training

Home

Experiments

Model registry

registered models

## random-forest-classifier

Created Time: 08/02/2026, 01:57:01 PM

Last Modified: 08/02/2026, 02:04:13 PM

> Description [Edit](#)

> Tags

▼ Versions [Compare](#)

New model registry UI

|   | Version                   | Registered at ↕         | Created by | Tags                | Aliases             | Description |
|---|---------------------------|-------------------------|------------|---------------------|---------------------|-------------|
| ✓ | <a href="#">Version 3</a> | 08/02/2026, 02:04:13 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |             |
| ✓ | <a href="#">Version 2</a> | 08/02/2026, 01:59:59 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |             |
| ✓ | <a href="#">Version 1</a> | 08/02/2026, 01:57:01 PM |            | <a href="#">Add</a> | <a href="#">Add</a> |             |

< Previous

Next >

25 / page





# source

- <https://medium.com/@faizulkhan56/mastering-mlflow-model-registry-a-complete-hands-on-guide-1309beeac333>